

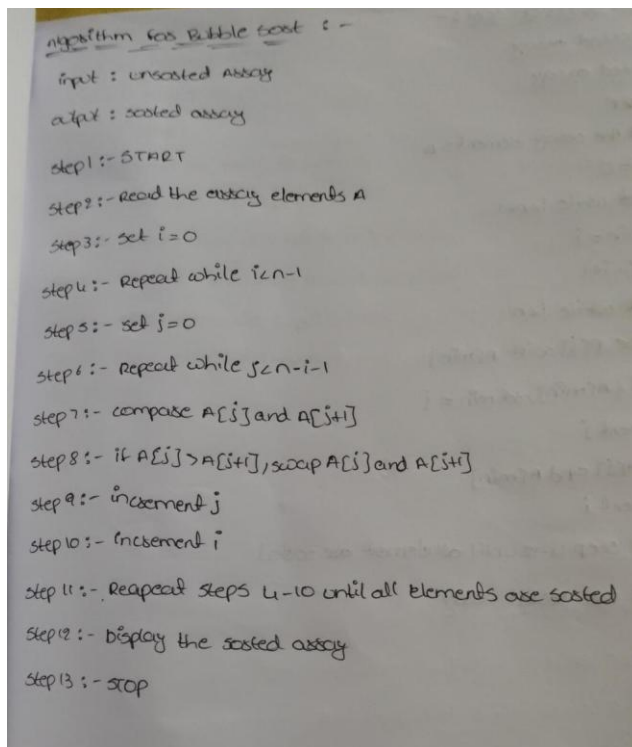
PRACTICAL-01

AIM:

Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

BUBBLE SORT:

ALGORITHM:



CODE:

```
#include <iostream>
using namespace std;
class bubblesort
{
public:
    int size, arr[100], i;
    void arrayinput()
    {
        cout << "Enter array size: ";
```

```
    cin >> size;
    cout << "Enter array elements:\n";
    for (i = 0; i < size; i++)
    {
        cin >> arr[i];
    }
}
void printarray()
{
    for (i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}
void bubblelogic()
{
    int j, temp;
    for (i = 0; i < size - 1; i++)
    {
        for (j = 0; j < size - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
};
int main()
{
    bubblesort bs;
    bs.arrayinput();
    cout << "Before sorting: ";
    bs.printarray();
    bs.bubblelogic();
    cout << "After sorting: ";
    bs.printarray();
    return 0;
}
```

OUTPUT:

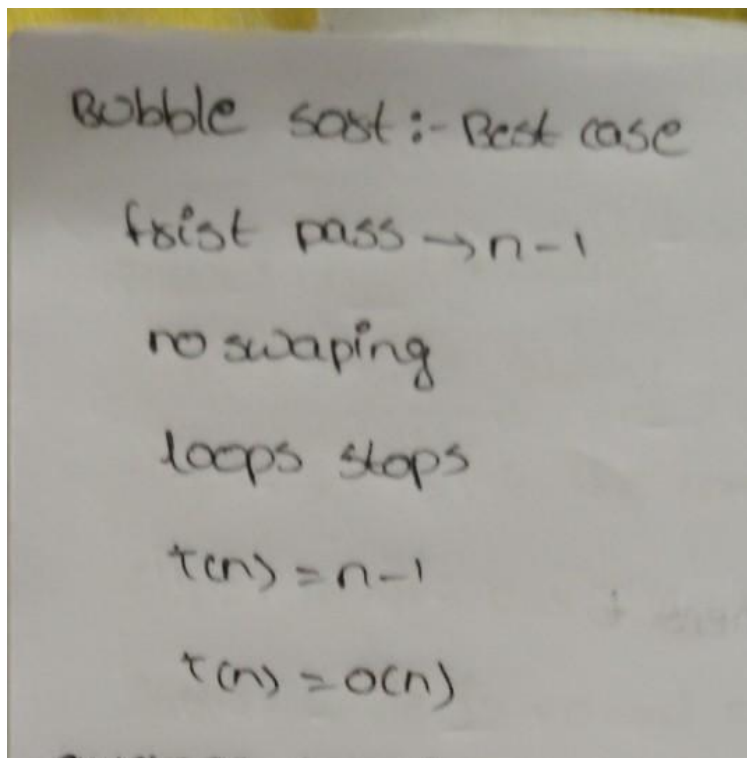
```
Output

Enter array size: 4
Enter array elements:
22
59
20
54
Before sorting: 22 59 20 54
After sorting: 20 22 54 59

=== Code Execution Successful ===
```

TIME COMPLEXITY:

Best case: if the given is already sorted

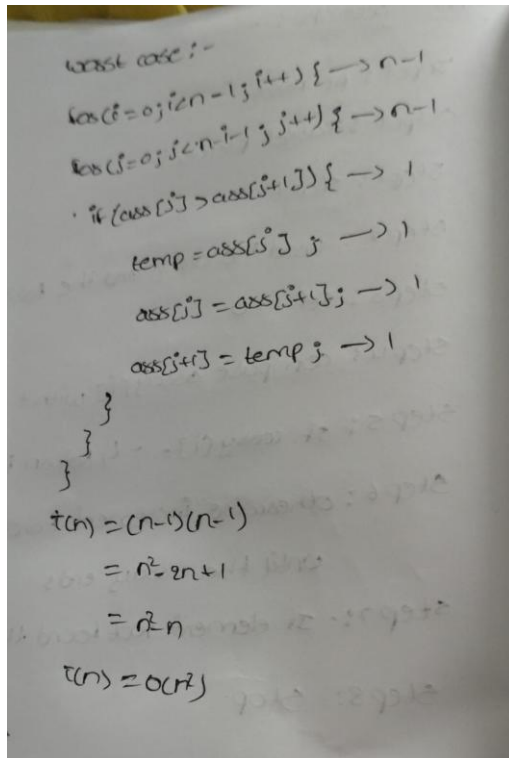


Average case:

average case :-
for (i=0; i<n-1; i++) { $\rightarrow n-1$
 for (j=0; j<n-i-1; j++) { $\rightarrow n-1$
 if (arr[j] > arr[j+1]) { $\rightarrow 1$
 temp = arr[j]; $\rightarrow 1$
 arr[j] = arr[j+1]; $\rightarrow 1$
 arr[j+1] = temp; $\rightarrow 1$
 }
 }
}

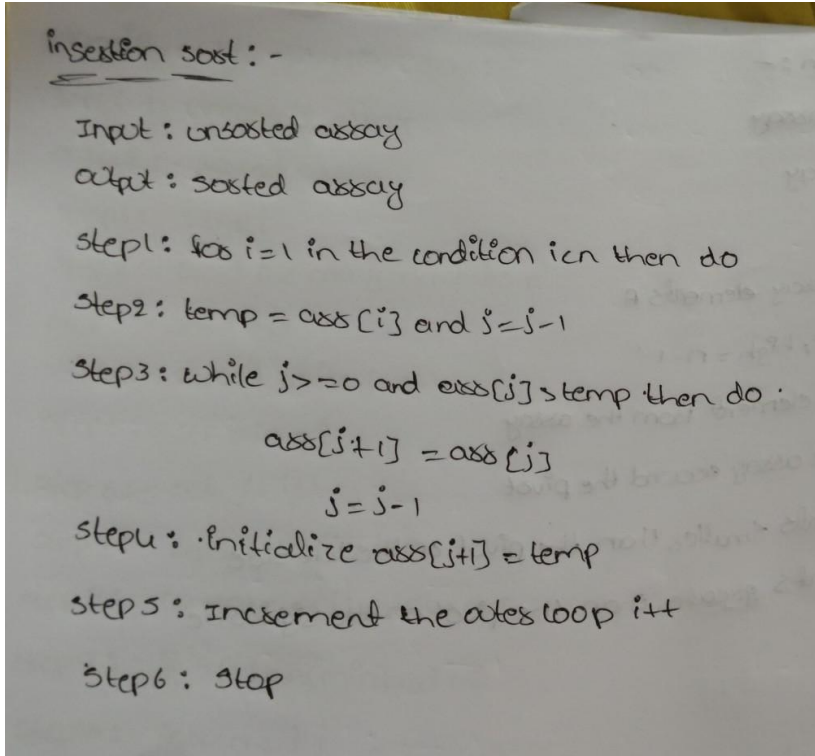
$$T(n) = (n-1)(n-1)$$
$$= n^2 - 2n + 1$$
$$= n^2 - n$$
$$T(n) = O(n^2)$$

Worst case : if array is in reverse order it takes many comparisons and swaps



INSERTION SORT:

ALGORITHM:



Insertion sort :-
Input : unsorted array
Output : sorted array
Step1 : for $i=1$ in the condition $i < n$ then do
Step2 : $temp = arr[i]$ and $j = j-1$
Step3 : while $j >= 0$ and $arr[j] > temp$ then do :
 $arr[j+1] = arr[j]$
 $j = j-1$
Step4 : initialize $arr[j+1] = temp$
Step5 : Increment the outer loop $i++$
Step6 : stop

CODE:

```
#include <iostream>
using namespace std;

class InsertionSort
{
public:
    int size, arr[100], i, j, temp;

    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;

        cout << "Enter array elements: ";
        for (i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }
}
```

```
void printarray()
{
    for (i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void insertionsortlogic()
{
    for (i = 1; i < size; i++)
    {
        temp = arr[i];
        j = i - 1;

        while (j >= 0 && arr[j] > temp)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = temp;
    }
}

};

int main()
{
    InsertionSort is;

    is.arrayinput();

    cout << "Before sorting: ";
    is.printarray();

    is.insertionsortlogic();

    cout << "After sorting: ";
    is.printarray();
    return 0;
}
```

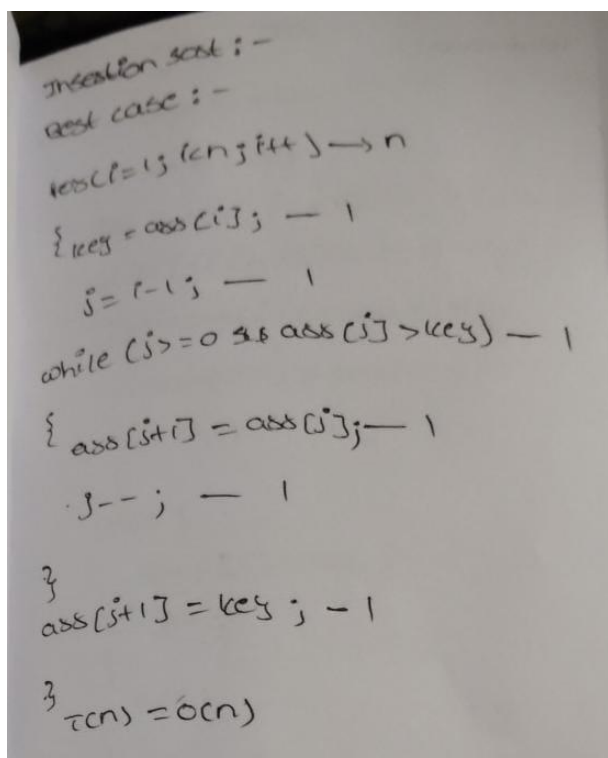
OUTPUT:

```
Output
Enter array size: 3
Enter array elements: 54
21
78
Before sorting: 54 21 78
After sorting: 21 54 78

=== Code Execution Successful ===
```

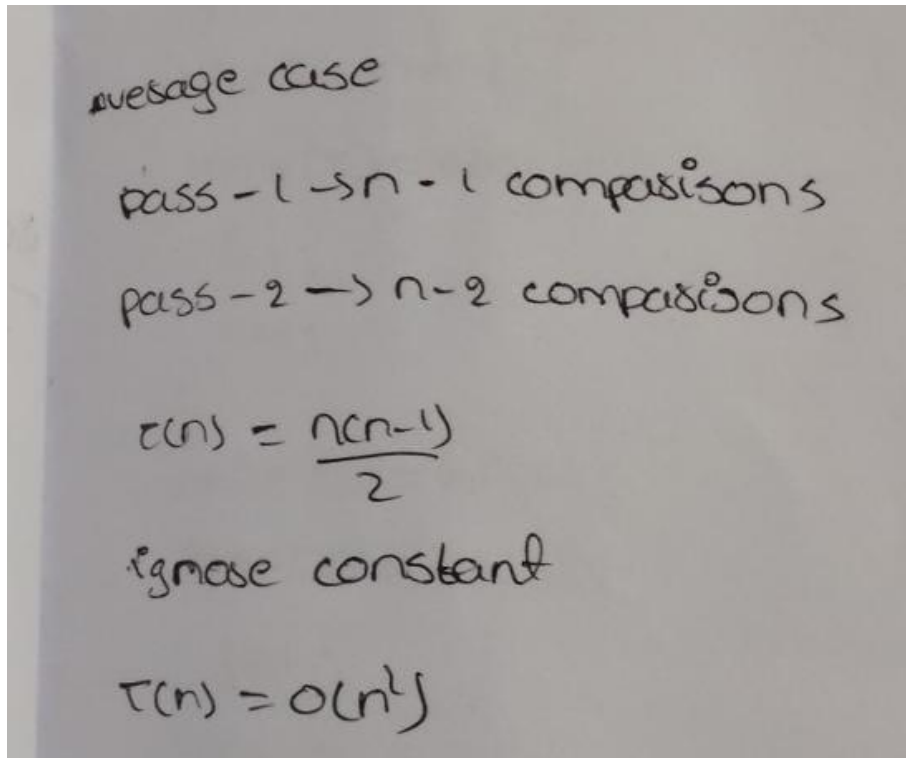
TIME COMPLEXITY:

Best case: if array is sorted we need only one comparison

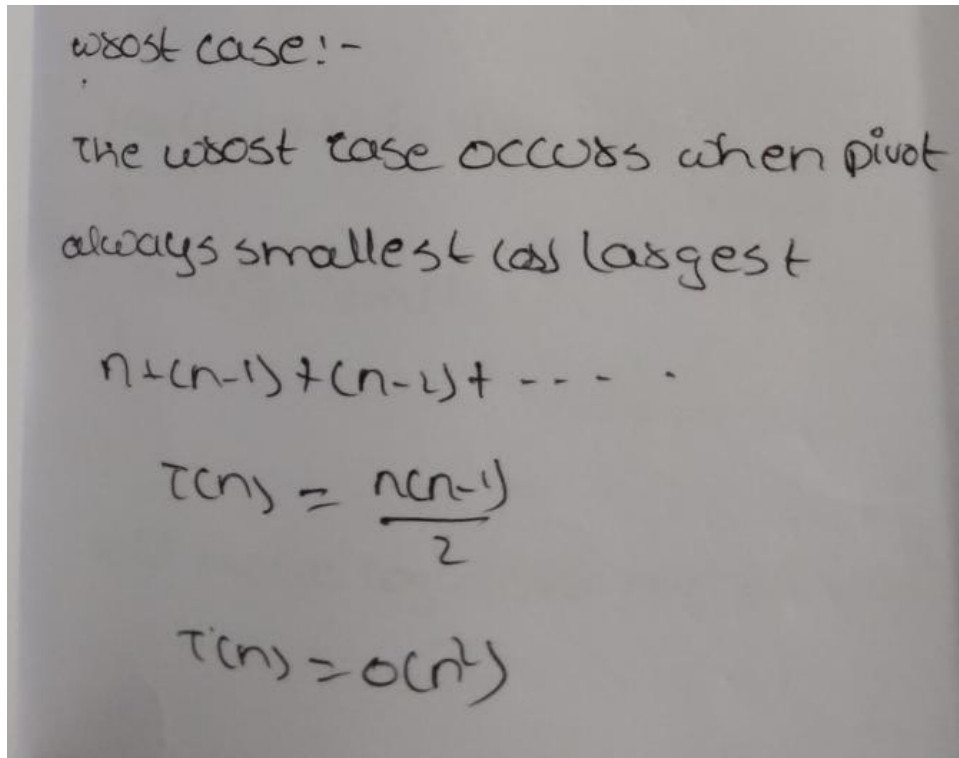


Insertion sort :-
Best case :-
key = arr[i] (arr[i+1]) $\rightarrow n$
{ key = arr[i]; $\rightarrow 1$
j = i-1; $\rightarrow 1$
while (j >= 0 && arr[j] > key) $\rightarrow 1$
{ arr[j+1] = arr[j]; $\rightarrow 1$
j--; $\rightarrow 1$
}
arr[j+1] = key; $\rightarrow 1$
}
T(n) = O(n)

Average case:

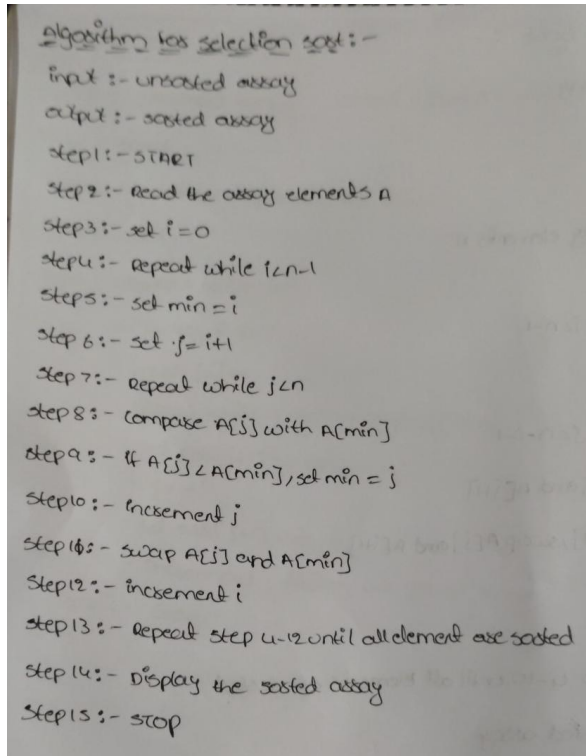


Worst case:



SELECTION SORT:

ALGORITHM:



Algorithm for selection sort:-
input :- unsorted array
output :- sorted array
step1:- START
step2:- Read the array elements A
step3:- set $i = 0$
step4:- Repeat while $i < n-1$
step5:- set $min = i$
step6:- set $j = i+1$
step7:- Repeat while $j < n$
step8:- Compare $A[j]$ with $A[min]$
step9:- If $A[j] < A[min]$, set $min = j$
step10:- Increment j
step11:- Swap $A[j]$ and $A[min]$
step12:- Increment i
step13:- Repeat step 4-12 until all elements are sorted
step14:- Display the sorted array
step15:- STOP

CODE:

```
#include <iostream>
using namespace std;

class SelectionSort
{
public:
    int size, arr[100], i, j, temp;

    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;

        cout << "Enter array elements: ";
        for (i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }
}
```

```
void printarray()
{
    for (i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void selectionsortlogic()
{
    for (i = 0; i < size - 1; i++)
    {
        int min = i;

        for (j = i + 1; j < size; j++)
        {
            if (arr[j] < arr[min])
            {
                min = j;
            }
        }

        if (min != i)
        {
            temp = arr[i];
            arr[i] = arr[min];
            arr[min] = temp;
        }
    }
}

};

int main()
{
    SelectionSort ss;

    ss.arrayinput();

    cout << "Before sorting: ";
    ss.printarray();

    ss.selectionsortlogic();

    cout << "After sorting: ";
```

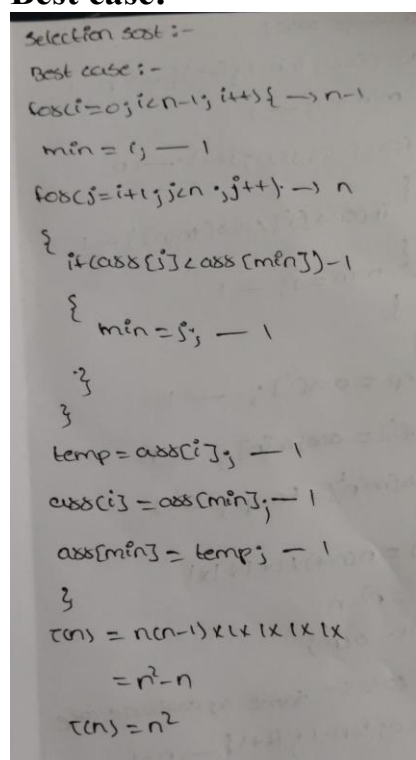
```
ss.printarray();  
    return 0;  
}
```

OUTPUT:

```
Output  
Enter array size: 3  
Enter array elements: 21  
65  
  
3  
Before sorting: 21 65 3  
After sorting: 3 21 65  
  
=== Code Execution Successful ===
```

TIME COMPLEXITY:

Best case:



selection sort :-
Best case :-
for (i=0; i<n-1; i++) { $\rightarrow n-1$
 min = i; $\rightarrow 1$
 for (j=i+1; j<n; j++) $\rightarrow n$
 {
 if (arr[j] < arr[min]) $\rightarrow 1$
 {
 min = j; $\rightarrow 1$
 }
 }
 temp = arr[i]; $\rightarrow 1$
 arr[i] = arr[min]; $\rightarrow 1$
 arr[min] = temp; $\rightarrow 1$
}
 $T(n) = (n-1) \times 1 \times 1 \times 1 \times 1$
 $= n^2 - n$
 $T(n) = n^2$

Average case:

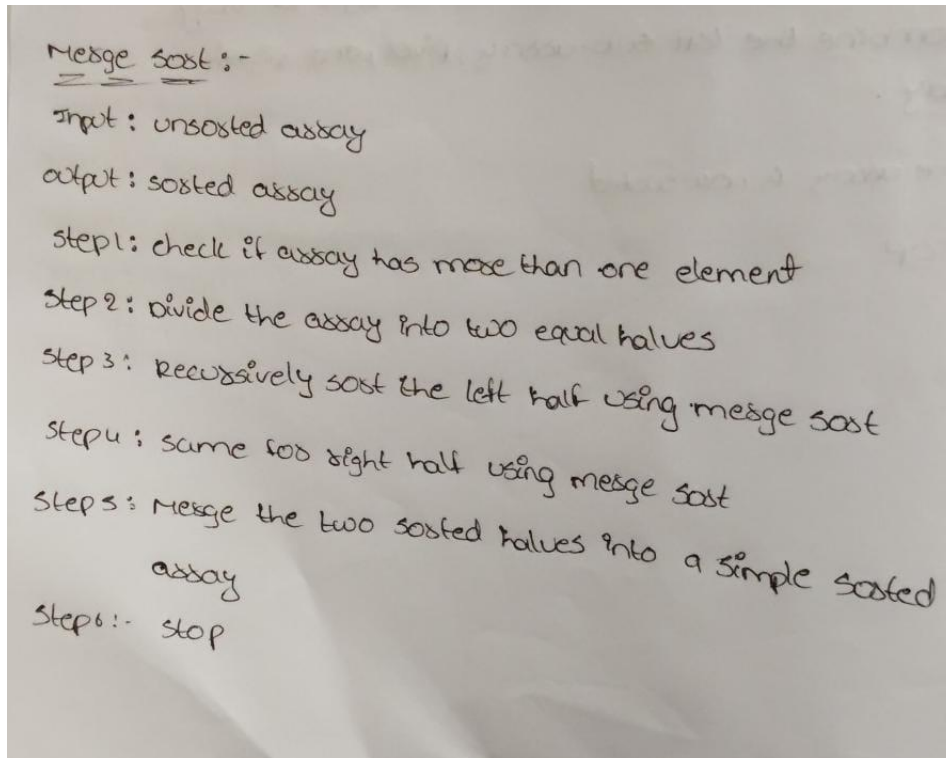
```
average case:
for (i=0; i<n-1; i++) { → n-1
    min = i; → 1
    for (j=0; j<n; j++) → n
    {
        if (arr[j] < arr[min]) → 1
        {
            min = j; → 1
        }
    }
    temp = arr[i]; → 1
    arr[i] = arr[min]; → 1
    arr[min] = temp; → 1
}
T(n) = n(n-1) + 1 + 1 + 1 + 1
      = n2 - n
      T(n) = O(n2)
```

Worst case:

```
worst case: - same as average case
for (i=0; i<n-1; i++) { → n-1
    min = i; → 1
    for (j=0; j<n; j++) → n
    {
        if (arr[j] < arr[min]) → 1
        {
            min = j; → 1
        }
    }
    temp = arr[i]; → 1
    arr[i] = arr[min]; → 1
    arr[min] = temp; → 1
}
```

MERGE SORT:

ALGORITHM:



CODE:

```
#include <iostream>
using namespace std;

class mergesort
{
public:
    int size, arr[100], i;

    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;
        cout << "Enter array elements:\n";
        for (i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }
}
```

```
void printarray()
{
    for (i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void merge(int low, int mid, int high)
{
    int temp[100];
    int i = low, j = mid + 1, k = low;

    while (i <= mid && j <= high)
    {
        if (arr[i] <= arr[j])
        {
            temp[k] = arr[i];
            i++;
        }
        else
        {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }

    while (i <= mid)
    {
        temp[k] = arr[i];
        i++;
        k++;
    }

    while (j <= high)
    {
        temp[k] = arr[j];
        j++;
        k++;
    }
}
```

```
        for (i = low; i <= high; i++)
        {
            arr[i] = temp[i];
        }
    }

    void mergellogic(int low, int high)
    {
        if (low < high)
        {
            int mid = (low + high) / 2;

            mergellogic(low, mid);
            mergellogic(mid + 1, high);

            merge(low, mid, high);
        }
    }
};

int main()
{
    mergesort ms;

    ms.arrayinput();

    cout << "Before sorting: ";
    ms.printarray();

    ms.mergellogic(0, ms.size - 1);

    cout << "After sorting: ";
    ms.printarray();

    return 0;
}
```

OUTPUT:

```
Enter array size: 3
Enter array elements:
21
5
69
Before sorting: 21 5 69
After sorting: 5 21 69

=== Code Execution Successful ===
```

TIME COMPLEXITY:

Best case, Average case and Worst case:

```
merge sort :-  
void merge (int low, int mid, int high)  
{  
    int temp[100];  
    int i = low, j = mid + 1, k = low;  
    while (i <= mid && j <= high){  
        if (arr[i] <= arr[j]){  
            temp[k] = arr[i];  
            i++;  
        }  
        else {  
            temp[k] = arr[j];  
            j++;  
        }  
        k++;  
    }  
    while (i <= mid){  
        temp[k] = arr[i];  
        i++;  
        k++;  
    }  
    while (j <= high){  
        temp[k] = arr[j];  
        j++;  
        k++;  
    }  
    for (i = low; i <= high; i++)  
    {  
        arr[i] = temp[i];  
    }  
}
```

$$T(n) = T(\frac{n}{2}) + T(\frac{n}{2}) + n$$

$$T(n) = 2T(\frac{n}{2}) + n$$

$$T(\frac{n}{2}) = 2T(\frac{n}{4}) + \frac{n}{2}$$

$$T(n) = 4T(\frac{n}{4}) + 2n$$

$$T(n) = 8T(\frac{n}{8}) + 3n$$

$$T(n) = 16T(\frac{n}{16}) + 4n$$

$$T(n) = 32T(\frac{n}{32}) + 5n$$

$$T(n) = 64T(\frac{n}{64}) + 6n$$

$$T(n) = 128T(\frac{n}{128}) + 7n$$

$$T(n) = 256T(\frac{n}{256}) + 8n$$

$$T(n) = 512T(\frac{n}{512}) + 9n$$

$$T(n) = 1024T(\frac{n}{1024}) + 10n$$

$$T(n) = 2048T(\frac{n}{2048}) + 11n$$

$$T(n) = 4096T(\frac{n}{4096}) + 12n$$

$$T(n) = 8192T(\frac{n}{8192}) + 13n$$

$$T(n) = 16384T(\frac{n}{16384}) + 14n$$

$$T(n) = 32768T(\frac{n}{32768}) + 15n$$

$$T(n) = 65536T(\frac{n}{65536}) + 16n$$

$$T(n) = 131072T(\frac{n}{131072}) + 17n$$

$$T(n) = 262144T(\frac{n}{262144}) + 18n$$

$$T(n) = 524288T(\frac{n}{524288}) + 19n$$

$$T(n) = 1048576T(\frac{n}{1048576}) + 20n$$

$$T(n) = 2097152T(\frac{n}{2097152}) + 21n$$

$$T(n) = 4194304T(\frac{n}{4194304}) + 22n$$

$$T(n) = 8388608T(\frac{n}{8388608}) + 23n$$

$$T(n) = 16777216T(\frac{n}{16777216}) + 24n$$

$$T(n) = 33554432T(\frac{n}{33554432}) + 25n$$

$$T(n) = 67108864T(\frac{n}{67108864}) + 26n$$

$$T(n) = 134217728T(\frac{n}{134217728}) + 27n$$

$$T(n) = 268435456T(\frac{n}{268435456}) + 28n$$

$$T(n) = 536870912T(\frac{n}{536870912}) + 29n$$

$$T(n) = 1073741824T(\frac{n}{1073741824}) + 30n$$

$$T(n) = 2147483648T(\frac{n}{2147483648}) + 31n$$

$$T(n) = 4294967296T(\frac{n}{4294967296}) + 32n$$

$$T(n) = 8589934592T(\frac{n}{8589934592}) + 33n$$

$$T(n) = 17179869184T(\frac{n}{17179869184}) + 34n$$

$$T(n) = 34359738368T(\frac{n}{34359738368}) + 35n$$

$$T(n) = 68719476736T(\frac{n}{68719476736}) + 36n$$

$$T(n) = 137438953472T(\frac{n}{137438953472}) + 37n$$

$$T(n) = 274877906944T(\frac{n}{274877906944}) + 38n$$

$$T(n) = 549755813888T(\frac{n}{549755813888}) + 39n$$

$$T(n) = 1099511627776T(\frac{n}{1099511627776}) + 40n$$

$$T(n) = 2199023255552T(\frac{n}{2199023255552}) + 41n$$

$$T(n) = 4398046511104T(\frac{n}{4398046511104}) + 42n$$

$$T(n) = 8796093022208T(\frac{n}{8796093022208}) + 43n$$

$$T(n) = 17592186044416T(\frac{n}{17592186044416}) + 44n$$

$$T(n) = 35184372088832T(\frac{n}{35184372088832}) + 45n$$

$$T(n) = 70368744177664T(\frac{n}{70368744177664}) + 46n$$

$$T(n) = 140737488355328T(\frac{n}{140737488355328}) + 47n$$

$$T(n) = 281474976710656T(\frac{n}{281474976710656}) + 48n$$

$$T(n) = 562949953421312T(\frac{n}{562949953421312}) + 49n$$

$$T(n) = 1125899906842624T(\frac{n}{1125899906842624}) + 50n$$

$$T(n) = 2251799813685248T(\frac{n}{2251799813685248}) + 51n$$

$$T(n) = 4503599627370496T(\frac{n}{4503599627370496}) + 52n$$

$$T(n) = 9007199254740992T(\frac{n}{9007199254740992}) + 53n$$

$$T(n) = 18014398509481984T(\frac{n}{18014398509481984}) + 54n$$

$$T(n) = 36028797018963968T(\frac{n}{36028797018963968}) + 55n$$

$$T(n) = 72057594037927936T(\frac{n}{72057594037927936}) + 56n$$

$$T(n) = 144115188075855872T(\frac{n}{144115188075855872}) + 57n$$

$$T(n) = 288230376151711744T(\frac{n}{288230376151711744}) + 58n$$

$$T(n) = 576460752303423488T(\frac{n}{576460752303423488}) + 59n$$

$$T(n) = 1152921504606846976T(\frac{n}{1152921504606846976}) + 60n$$

$$T(n) = 2305843009213693952T(\frac{n}{2305843009213693952}) + 61n$$

$$T(n) = 4611686018427387904T(\frac{n}{4611686018427387904}) + 62n$$

$$T(n) = 9223372036854775808T(\frac{n}{9223372036854775808}) + 63n$$

$$T(n) = 18446744073709551616T(\frac{n}{18446744073709551616}) + 64n$$

$$T(n) = 36893488147419103232T(\frac{n}{36893488147419103232}) + 65n$$

$$T(n) = 73786976294838206464T(\frac{n}{73786976294838206464}) + 66n$$

$$T(n) = 147573952589676412928T(\frac{n}{147573952589676412928}) + 67n$$

$$T(n) = 295147905179352825856T(\frac{n}{295147905179352825856}) + 68n$$

$$T(n) = 590295810358705651712T(\frac{n}{590295810358705651712}) + 69n$$

$$T(n) = 1180591620717411303424T(\frac{n}{1180591620717411303424}) + 70n$$

$$T(n) = 2361183241434822606848T(\frac{n}{2361183241434822606848}) + 71n$$

$$T(n) = 4722366482869645213696T(\frac{n}{4722366482869645213696}) + 72n$$

$$T(n) = 9444732965739290427392T(\frac{n}{9444732965739290427392}) + 73n$$

$$T(n) = 18889465931478580854784T(\frac{n}{18889465931478580854784}) + 74n$$

$$T(n) = 37778931862957161709568T(\frac{n}{37778931862957161709568}) + 75n$$

$$T(n) = 75557863725914323419136T(\frac{n}{75557863725914323419136}) + 76n$$

$$T(n) = 151115727451828646838272T(\frac{n}{151115727451828646838272}) + 77n$$

$$T(n) = 302231454903657293676544T(\frac{n}{302231454903657293676544}) + 78n$$

$$T(n) = 604462909807314587353088T(\frac{n}{604462909807314587353088}) + 79n$$

$$T(n) = 1208925819614629174706176T(\frac{n}{1208925819614629174706176}) + 80n$$

$$T(n) = 2417851639229258349412352T(\frac{n}{2417851639229258349412352}) + 81n$$

$$T(n) = 4835703278458516698824704T(\frac{n}{4835703278458516698824704}) + 82n$$

$$T(n) = 9671406556917033397649408T(\frac{n}{9671406556917033397649408}) + 83n$$

$$T(n) = 19342813113834066795298816T(\frac{n}{19342813113834066795298816}) + 84n$$

$$T(n) = 38685626227668133590597632T(\frac{n}{38685626227668133590597632}) + 85n$$

$$T(n) = 77371252455336267181195264T(\frac{n}{77371252455336267181195264}) + 86n$$

$$T(n) = 154742504910672534362390528T(\frac{n}{154742504910672534362390528}) + 87n$$

$$T(n) = 309485009821345068724781056T(\frac{n}{309485009821345068724781056}) + 88n$$

$$T(n) = 618970019642690137449562112T(\frac{n}{618970019642690137449562112}) + 89n$$

$$T(n) = 1237940039285380274899124224T(\frac{n}{1237940039285380274899124224}) + 90n$$

$$T(n) = 2475880078570760549798248448T(\frac{n}{2475880078570760549798248448}) + 91n$$

$$T(n) = 4951760157141521099596496896T(\frac{n}{4951760157141521099596496896}) + 92n$$

$$T(n) = 9903520314283042199192993792T(\frac{n}{9903520314283042199192993792}) + 93n$$

$$T(n) = 19807040628566084398385987584T(\frac{n}{19807040628566084398385987584}) + 94n$$

$$T(n) = 39614081257132168796771975168T(\frac{n}{39614081257132168796771975168}) + 95n$$

$$T(n) = 79228162514264337593543950336T(\frac{n}{79228162514264337593543950336}) + 96n$$

$$T(n) = 158456325028528675187087900672T(\frac{n}{158456325028528675187087900672}) + 97n$$

$$T(n) = 316912650057057350374175801344T(\frac{n}{316912650057057350374175801344}) + 98n$$

$$T(n) = 633825300114114700748351602688T(\frac{n}{633825300114114700748351602688}) + 99n$$

$$T(n) = 1267650600228229401496703205376T(\frac{n}{1267650600228229401496703205376}) + 100n$$

$$T(n) = 2535301200456458802993406410752T(\frac{n}{2535301200456458802993406410752}) + 101n$$

$$T(n) = 5070602400912917605986812821504T(\frac{n}{5070602400912917605986812821504}) + 102n$$

$$T(n) = 10141204801825835211973625643008T(\frac{n}{10141204801825835211973625643008}) + 103n$$

$$T(n) = 20282409603651670423947251286016T(\frac{n}{20282409603651670423947251286016}) + 104n$$

$$T(n) = 40564819207303340847894502572032T(\frac{n}{40564819207303340847894502572032}) + 105n$$

$$T(n) = 81129638414606681695789005144064T(\frac{n}{81129638414606681695789005144064}) + 106n$$

$$T(n) = 162259276829213363391578010288128T(\frac{n}{162259276829213363391578010288128}) + 107n$$

$$T(n) = 324518553658426726783156020576256T(\frac{n}{324518553658426726783156020576256}) + 108n$$

$$T(n) = 649037107316853453566312041152512T(\frac{n}{649037107316853453566312041152512}) + 109n$$

$$T(n) = 1298074214633706907132624082305024T(\frac{n}{1298074214633706907132624082305024}) + 110n$$

$$T(n) = 2596148429267413814265248164610048T(\frac{n}{2596148429267413814265248164610048}) + 111n$$

$$T(n) = 5192296858534827628530496329220096T(\frac{n}{5192296858534827628530496329220096}) + 112n$$

$$T(n) = 10384593717069655257060992658440192T(\frac{n}{10384593717069655257060992658440192}) + 113n$$

$$T(n) = 20769187434139310514121985316880384T(\frac{n}{20769187434139310514121985316880384}) + 114n$$

$$T(n) = 41538374868278621028243970633760768T(\frac{n}{41538374868278621028243970633760768}) + 115n$$

$$T(n) = 83076749736557242056487941267521536T(\frac{n}{83076749736557242056487941267521536}) + 116n$$

$$T(n) = 166153499473114484112975882535043072T(\frac{n}{166153499473114484112975882535043072}) + 117n$$

$$T(n) = 332306998946228968225951765070086144T(\frac{n}{332306998946228968225951765070086144}) + 118n$$

$$T(n) = 664613997892457936451903530140172288T(\frac{n}{664613997892457936451903530140172288}) + 119n$$

$$T(n) = 1329227995784915872903807060280344576T(\frac{n}{1329227995784915872903807060280344576}) + 120n$$

$$T(n) = 2658455991569831745807614120560689152T(\frac{n}{2658455991569831745807614120560689152}) + 121n$$

$$T(n) = 5316911983139663491615228241121378304T(\frac{n}{5316911983139663491615228241121378304}) + 122n$$

$$T(n) = 10633823966279326983230456482242756608T(\frac{n}{10633823966279326983230456482242756608}) + 123n$$

$$T(n) = 21267647932558653966460912964485513216T(\frac{n}{21267647932558653966460912964485513216}) + 124n$$

$$T(n) = 42535295865117307932921825928971026432T(\frac{n}{42535295865117307932921825928971026432}) + 125n$$

$$T(n) = 85070591730234615865843651857942052864T(\frac{n}{85070591730234615865843651857942052864}) + 126n$$

$$T(n) = 170141183460469231731687303715884105728T(\frac{n}{170141183460469231731687303715884105728}) + 127n$$

$$T(n) = 340282366920938463463374607431768211456T(\frac{n}{340282366920938463463374607431768211456}) + 128n$$

$$T(n) = 680564733841876926926749214863536422912T(\frac{n}{680564733841876926926749214863536422912}) + 129n$$

$$T(n) = 1361129467683753853853498429727072845824T(\frac{n}{1361129467683753853853498429727072845824}) + 130n$$

$$T(n) = 2722258935367507707706996859454145691648T(\frac{n}{2722258935367507707706996859454145691648}) + 131n$$

$$T(n) = 5444517870735015415413993718908291383296T(\frac{n}{5444517870735015415413993718908291383296}) + 132n$$

$$T(n) = 10889035741470030830827987437816582766592T(\frac{n}{10889035741470030830827987437816582766592}) + 133n$$

$$T(n) = 21778071482940061661655974875633165533184T(\frac{n}{21778071482940061661655974875633165533184}) + 134n$$

$$T(n) = 43556142965880123323311949751266331066368T(\frac{n}{43556142965880123323311949751266331066368}) + 135n$$

$$T(n) = 87112285931760246646623899502532662132736T(\frac{n}{87112285931760246646623899502532662132736}) + 136n$$

$$T(n) = 174224571863520493293247799005065324265472T(\frac{n}{174224571863520493293247799005065324265472}) + 137n$$

$$T(n) = 348449143727040986586495598010130648530944T(\frac{n}{348449143727040986586495598010130648530944}) + 138n$$

$$T(n) = 696898287454081973172991196020261297061888T(\frac{n}{696898287454081973172991196020261297061888}) + 139n$$

$$T(n) = 1393796574908163946345982392040522594123776T(\frac{n}{1393796574908163946345982392040522594123776}) + 140n$$

$$T(n) = 2787593149816327892691964784081045188247552T(\frac{n}{2787593149816327892691964784081045188247552}) + 141n$$

$$T(n) = 5575186299632655785383929568162090376495104T(\frac{n}{5575186299632655785383929568162090376495104}) + 142n$$

$$T(n) = 11150372599265311570767859136324180752990208T(\frac{n}{11150372599265311570767859136324180752990208}) + 143n$$

$$T(n) = 22300745198530623141535718272648361505980416T(\frac{n}{22300745198530623141535718272648361505980416}) + 144n$$

$$T(n) = 44601490397061246283071436545296723011960832T(\frac{n}{44601490397061246283071436545296723011960832}) + 145n$$

$$T(n) = 89202980794122492566142873090593446023921664T(\frac{n}{89202980794122492566142873090593446023921664}) + 146n$$

$$T(n) = 178405961588244985132285746181186892047843328T(\frac{n}{178405961588244985132285746181186892047843328}) + 147n$$

$$T(n) = 356811923176489970264571492362373784095686656T(\frac{n}{356811923176489970264571492362373784095686656}) + 148n$$

$$T(n) = 713623846352979940529142984724747568191373312T(\frac{n}{713623846352979940529142984724747568191373312}) + 149n$$

$$T(n) = 1427247692705959881058285969449495136382746624T(\frac{n}{1427247692705959881058285969449495136382746624}) + 150n$$

$$T(n) = 2854495385411919762116571938898990272765493248T(\frac{n}{2854495385411919762116571938898990272765493248}) + 151n$$

$$T(n) = 5708990770823839524233143877797980545530986496T(\frac{n}{5708990770823839524233143877797980545530986496}) + 152n$$

$$T(n) = 11417981541647679048466287755595961091061972992T(\frac{n}{11417981541647679048466287755595961091061972992}) + 153n$$

$$T(n) = 22835963083295358096932575511191922182123945984T(\frac{n}{22835963083295358096932575511191922182123945984}) + 154n$$

$$T(n) = 45671926166590716193865151022383844364247891968T(\frac{n}{45671926166590716193865151022383844364247891968}) + 155n$$

$$T(n) = 91343852333181432387730302044767688728495783936T(\frac{n}{91343852333181432387730302044767688728495783936}) + 156n$$

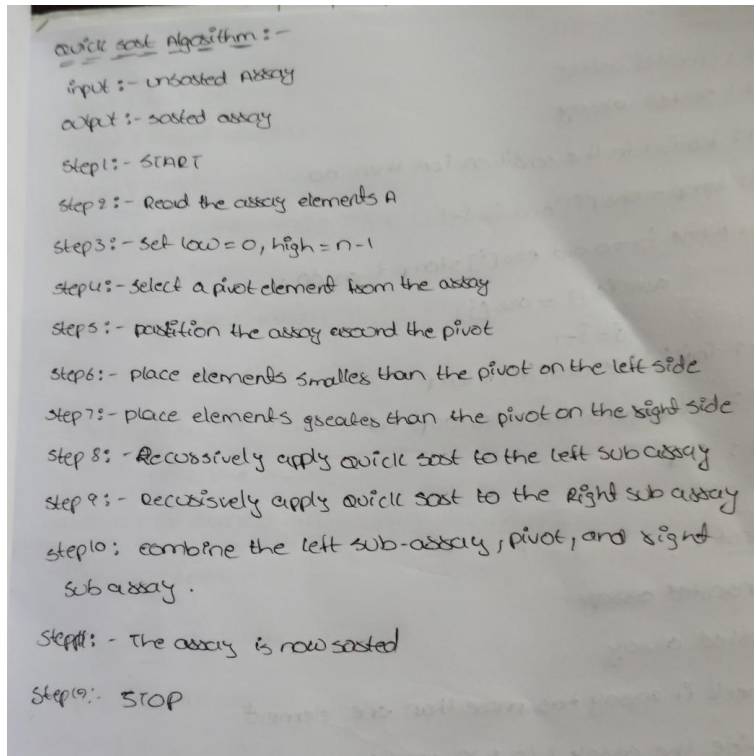
$$T(n) = 182687704666362864775460604089535377456991567872T(\frac{n}{182687704666362864775460604089535377456991567872}) + 157n$$

$$T(n) = 365375409332725729550921208179070754913983135744T(\frac{n}{365375409332725729550921208179070754913983135744}) + 158n$$

$$T(n) = 730750818665451459101842416358141509827966271488T(\frac{n}{730750818665451459101842416358141509827966271488}) + 159n$$

$$T(n) = 1461501637330902918203684832716283019655932542976T$$

ALGORITHM:



CODE:

```
#include <iostream>
#include <utility>
using namespace std;

class QuickSort
{
public:
    int size, arr[100];

    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;
        cout << "Enter array elements: ";
        for (int i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }

    void printarray()
```

```
{
    for (int i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int partition(int lb, int ub)
{
    int pivot = arr[lb];
    int start = lb;
    int end = ub;

    while (start < end)
    {
        while (start <= ub && arr[start] <= pivot)
            start++;

        while (arr[end] > pivot)
            end--;

        if (start < end)
            swap(arr[start], arr[end]);
    }

    swap(arr[lb], arr[end]);
    return end;
}

void quicksortlogic(int lb, int ub)
{
    if (lb < ub)
    {
        int loc = partition(lb, ub);
        quicksortlogic(lb, loc - 1);
        quicksortlogic(loc + 1, ub);
    }
}

void sort()
{
    quicksortlogic(0, size - 1);
}
```

```
    }  
};  
  
int main()  
{  
    QuickSort qs;  
  
    qs.arrayinput();  
  
    cout << "Before sorting: ";  
    qs.printarray();  
  
    qs.sort();  
  
    cout << "After sorting: ";  
    qs.printarray();  
  
    return 0;  
}
```

OUTPUT:

```
Output  
Enter array size: 3  
Enter array elements: 25  
65  
8  
Before sorting: 25 65 8  
After sorting: 8 25 65  
  
=== Code Execution Successful ===
```

TIME COMPLEXITY:



quick sort :-
Best case
if (low < high) - 1

```

{
    pivot = arr[high]; - 1
    i = low - 1; - 1
    for (j = low; j < high; j++) - n
    {
        if (arr[j] < pivot) - 1
        {
            i++;
            temp = arr[i]; - 1
            arr[i] = arr[j]; - 1
            arr[j] = temp; - 1
        }
    }
    temp = arr[i+1]; - 1
    arr[i+1] = arr[high]; - 1
    arr[high] = temp; - 1
    p = i+1;
    quick sort (arr, low, p-1); - 1
    quick sort (arr, p+1, high); - 1
}

```

$T(n) = 2T(n/2) + O(n) \quad \text{--- (1)}$
 $T(n) = 2T(n/4) + \frac{n}{2} \quad \text{--- (2)}$
 sub @ in (1)
 $T(n) = 2(2T(n/4) + \frac{n}{2}) + n$
 $T(n) = 4T(n/4) + n + n$
 $T(n) = 4T(n/4) + 2n$
 $T(n) = 2^k T(\frac{n}{2^k}) + 2kn$
 $\frac{n}{2^k} = 1 \quad n = 2^k$
 $\log n = \log 2^k$
 $\log n = k$
 $T(n) = 2(\log^2(n) + \log n)$
 $T(n) = n \log^2(n) + 2n \log n$
 $T(n) = n + n \log n$
 $T(n) = O(n \log n)$

Average case:

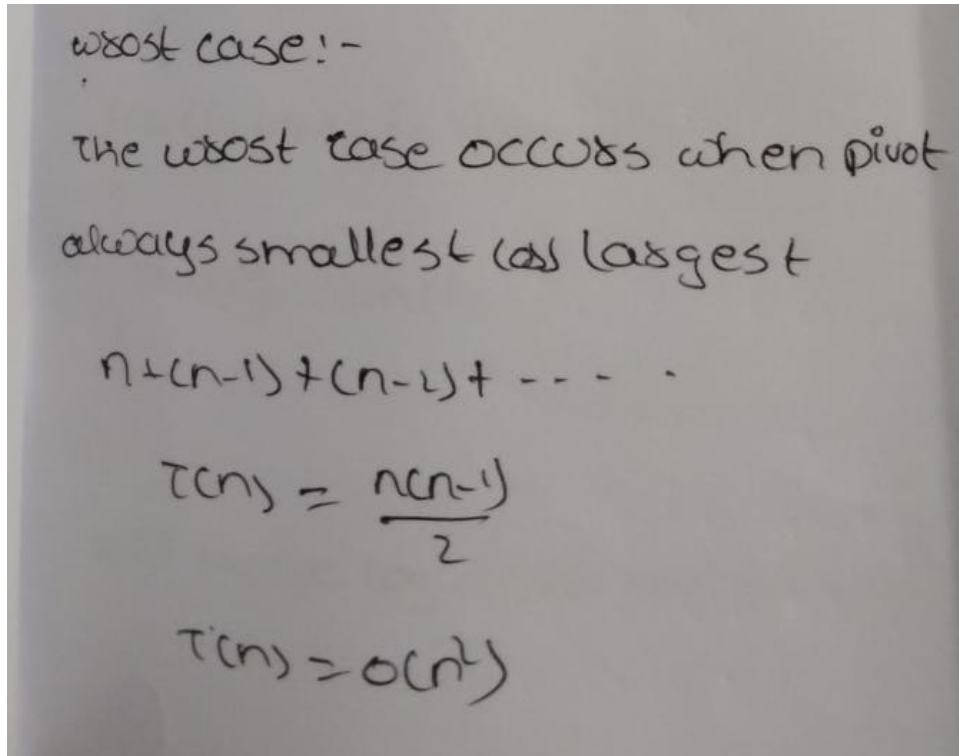
```

// Quick Sort
// (low, high)
{
    pivot = arr[high]; // 1
    i = low - 1; // 1
    for (j = low; j < high; j++) // n
    {
        if (arr[j] < pivot) // 1
        {
            i++;
            temp = arr[i]; // 1
            arr[i] = arr[j]; // 1
            arr[j] = temp; // 1
        }
    }
    temp = arr[i+1]; // 1
    arr[i+1] = arr[high]; // 1
    arr[high] = temp; // 1
    p = i+1;
    quickSort(arr, low, p-1); // 1
    quickSort(arr, p+1, high); // 1
}

```

$T(n) = 2T(n/2) + O(n) \quad \text{--- (1)}$
 $T(n) = 2T(n/2) + n \quad \text{--- (2)}$
 Sub (2) in (1)
 $T(n) = 2(2T(n/2) + n) + n$
 $T(n) = 4T(n/2) + n + n$
 $T(n) = 4T(n/2) + 2n$
 $T(n) = 2^k T(n/2^k) + 2kn$
 $\frac{n}{2^k} = 1 \quad n = 2^k$
 $\log n = \log 2^k$
 $\log n = k$
 $T(n) = 2(\log^2 n) + \log n$
 $T(n) = n \log^2 n + 1n \log n$
 $T(n) = n + n \log n$
 $T(n) = O(n \log n)$

Worst case:



Conclusion:

I understood that different sorting algorithms use different ways to arrange data. Bubble, Selection, Insertion, are easy to understand and implement, but they can take more time for large data. Merge sort, quick sort use a divide-and-conquer approach and are generally more efficient for large datasets. Overall, choosing an algorithm depends on the size and condition of the data and the require.